

1 Obsługa błędów

W większości przypadków **wywołanie systemowe** (funkcja systemowa) lub **funkcja biblioteczna** kończąc się **błędem** zwraca wartość -1 (czasami `NULL`) i przypisuje zmiennej zewnętrznej `errno` wartość wskazującą rodzaj błędu. Informacje o kodach błędów oraz odpowiadających im komunikatach można znaleźć w `man errno`. System sam z siebie nie zeruje nigdy zmiennej `errno`. Zatem niezerowa wartość tej zmiennej będzie zawsze zawierać kod ostatnio napotkanego błędu.

1.1 Funkcja biblioteczna perror

Pliki włączane	<stdio.h>		
Prototyp	void perror(const char *s);		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
			Nie

Funkcja `perror` wypisuje komunikat błędu poprzedzony napisem `*s` i znakiem `':'`.

→ UWAGA: Pliki nagłówkowe podaje się względem katalogu `/usr/include/`.

1.2 Funkcja biblioteczna strerror

Pliki włączane	<string.h>		
Prototyp	char *strerror(int errnum);		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	Wskaźnik do komunikatu błędu		Nie

Funkcja `strerror` odwzorowuje całkowitoliczbowy argument `errnum` (którym może być wartość `errno`) na komunikat błędu i zwraca wskaźnik do niego. Niektóre funkcje, np. do obsługi wątków standardu POSIX, nie ustawiają zmiennej `errno` w przypadku porażki, ale zwracają odpowiedni kod błędu. W takich przypadkach można użyć funkcji `strerror` do wypisania komunikatu odpowiadającego danemu kodowi błędu.

1.3 Użyteczne makra języka C

W obsłudze błędów oprócz informacji o przyczynie, zwracanej np. przez powyższe funkcje diagnostyczne, ważne jest także zlokalizowanie miejsca jego wystąpienia w kodzie źródłowym. Ułatwia to znacznie usuwanie błędów w programie, określane zwykle mianem debugowania (od ang. *debugging*). W standardzie języka C istnieją predefiniowane makra `__FILE__` i `__LINE__`, które zwracają odpowiednio nazwę pliku źródłowego (w formacie `%s`) i numer linii w tym pliku (w formacie `%d`) odnośnie miejsca ich wywołania. Od

standardu C99 dostępny jest ponadto predefiniowany identyfikator `__func__` zwracający nazwę funkcji (w formacie `%s`), w której się pojawia. Warto korzystać w powyższych udogodnień przy obsłudze błędów do wskazywania ich lokalizacji w programach.

Standard języka C udostępnia też specjalne makro o nazwie `assert` do pomocy w wykrywaniu błędów w kodzie:

```
#include <assert.h>
void assert(int expression);
```

Sprawdza ono w miejscu wywołania wartość wyrażenia `expression` i przerywa program, jeśli jego wynikiem jest 0 (czyli wartość logiczna `false`), a na standardowe wyjście błędów `stderr` przekazuje komunikat typu (może się różnić nieco w zależności od kompilatora):

```
program: file:line: function: Assertion 'expression' failed.
```

gdzie `program`, `file` i `function` oznaczają odpowiednio nazwy programu, pliku i funkcji, a `line` – numer linii w pliku kodu źródłowego, gdzie wywołanie `assert` zakończyło się niepowodzeniem. Jeżeli natomiast wartość `expression` jest niezerowa (logiczne `true`), to `assert` nie robi nic i program kontynuuje swoje działanie.

Przerwanie programu w `assert` następuje przy pomocy funkcji `abort`, która nie opróżnia buforów otwartych plików i nie wywołuje funkcji zarejestrowanych przez funkcję `atexit` (opisaną w następnym rozdziale). Ten fakt należy brać pod uwagę przy używaniu makra `assert`. Na ogół zaleca się korzystanie z niego w trakcie rozwoju programu w celu wykrywania błędów logicznych w kodzie. Po zakończeniu procesu debugowania można je wyłączyć dyrektywą preprocesora

```
#define NDEBUG
```

umieszczoną przed `#include <assert.h>` lub za pomocą opcji kompilatora `-DNDEBUG`.

Do obsługi błędów funkcji systemowych/bibliotecznych można napisać zestaw makr preprocesora korzystających z powyższych funkcji diagnostycznych oraz makr, jak to zostało zaproponowane w ćwiczeniu dodatkowym A1.

2 Procesy

PROGRAM: Nieaktywny, statyczny zbiór ułożonych w odpowiedniej kolejności instrukcji oraz towarzyszących im danych.

PROCES: *Podstawowe pojęcie w Uniksie.*

Abstrakcyjny twór składający się z wykonywanego (działającego) programu oraz bieżących danych o jego stanie i zasobach, za pomocą których system operacyjny steruje jego wykonywaniem. Proces jest jednostką dynamiczną. W Uniksie wiele **procesów** może być wykonywane **współbieżnie** (na jednym procesorze dzięki podziałowi czasu i przełączaniu kontekstu) – nazywa się to **wielozadaniowością**

→ UWAGA: **Proces** może wykonywać *co najwyżej jeden program*, natomiast ten sam **program** może być wykonywany w *dowolnej liczbie procesów*.

2.1 Identyfikatory związane z procesami

Podstawowe identyfikatory związane z procesami oraz funkcje systemowe służące do ich uzyskiwania przedstawione są w tabeli **1**. Identyfikatory te przyjmują wartości **liczb całkowitych nieujemnych**.

Pliki włączane		<sys/types.h>, <unistd.h>
Nazwa	Funkcja systemowa	Opis
UID	uid_t getuid(void);	identyfikator użytkownika (rzeczywisty)
GID	gid_t getgid(void);	identyfikator grupy użytkownika (rzeczywisty)
PID	pid_t getpid(void);	identyfikator procesu
PPID	pid_t getppid(void);	identyfikator procesu macierzystego (przodka)
PGID	pid_t getpgid(pid_t pid);	identyfikator grupy procesów (=PID lidera grupy)
	pid_t getpgrp(void);	≡ getpgid(0); PGID procesu bieżącego

Tabela 1: Identyfikatory związane z procesami oraz funkcje służące do ich uzyskiwania

kowitych nieujemnych. W Uniksie procesy mogą być łączone w grupy – każdy proces należący do takiej grupy posiada ten sam identyfikator grupy procesów PGID, który jest równy identyfikatorowi procesu PID lidera tej grupy. Spośród podanych funkcji jedynie **getpgid** może zakończyć się błędem – wówczas zwraca wartość -1 i ustawia zmienną **errno**.

→ UWAGA: W przypadku gdy kompilator zgłasza ostrzeżenie przy wywołaniu którejś z powyższych (lub poniższych) funkcji, należy sprawdzić jej opis w podręczniku systemowym **man** (np. **man getpgid**) i na początku pliku wstawić podane tam odpowiednie makro preprocesora. Makra te dotyczą zwykle używanej wersji biblioteki **glibc**, którą najprościej można sprawdzić wykonując z poziomu powłoki polecenie **ldd -version**.

Z poziomu powłoki podstawowe informacje o bieżących procesach można uzyskać przy pomocy komendy **ps**, np. **ps -el** podaje wykaz wszystkich bieżących procesów w tzw.

długim formacie (więcej szczegółów w podręczniku systemowym: `man ps`). Podgląd najbardziej aktywnych procesów w czasie rzeczywistym, wraz z wieloma szczegółami na temat używanych przez nie zasobów systemowych, można uzyskać za pomocą komendy `top`.

Pliki włączane	<sys/types.h>, <unistd.h>		
Prototyp	<code>pid_t setpgid(pid_t pid, pid_t pgrp);</code>		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	0	-1	Tak

Identyfikator grupy procesów PGID można zmienić. Do tego celu służy funkcja `setpgid` przedstawiona w powyższej tabeli. Umożliwia ona nadanie identyfikatorowi grupy procesów procesu lidera o identyfikatorze `pid` wartości `pgrp`. Jeżeli przekazana zostanie wartość `pid` równa zero, to zmieniony zostanie identyfikator PGID bieżącego procesu. Wartość `pgrp` odpowiada numerowi grupy, do której proces będzie od tej chwili należał. Jeśli wartością `pgrp` będzie zero, to proces o identyfikatorze równym `pid` stanie się liderem nowej grupy. Tak zwykle się dzieje kiedy uruchamiamy jakieś polecenie z poziomu powłoki Uniksa – wówczas tworzony jest nowy proces i dla niego wywoływana jest funkcja `setpgid` z wartością `pgrp` równą 0 i proces ten staje się liderem nowej grupy procesów. Funkcja ta może być skutecznie wywołana tylko kiedy wywołujący ją proces ma odpowiednie uprawnienia, np. jest procesem macierzystym procesu docelowego. W przypadku sukcesu, funkcja `setpgid` zwraca wartość 0, a w przypadku porażki wartość -1 i ustawia zmienną `errno` na odpowiedni kod błędu.

→ UWAGA: Nie należy mylić identyfikatora „grupy procesów” PGID z identyfikatorem „grupy użytkownika” procesu GID!

2.2 Tworzenie procesów potomnych – funkcja systemowa `fork`

Pliki włączane	<sys/types.h>, <unistd.h>		
Prototyp	<code>pid_t fork(void);</code>		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	→ 0 w procesie potomnym → PID procesu potomnego w procesie macierzystym	-1	Tak

Funkcja systemowa `fork` tworzy proces potomny, który jest kopią procesu macierzystego.

Typowe wywołanie funkcji fork

```

switch (fork())
{
    case -1:
        perror("fork error");
        exit(1);
    case 0:
        // akcja dla procesu potomnego
    default:
        // akcja dla procesu macierzystego, np. wywołanie funkcji wait
};

```

2.3 Kończenie działania procesu – funkcje exit i _exit

Pliki włączane	<stdlib.h>		
Prototyp	void exit(int status);		
Zwracana wartość	Sukces	Porażka	Czy zmienia errno
			Nie

Jednym ze sposobów zakończenia procesu jest wywołanie funkcji bibliotecznej `exit`. Funkcja ta wykonuje operacje zakończenia działania procesu i zwraca do procesu macierzystego całkowitoliczbową wartość `status`, oznaczającą status zakończenia procesu. Zgodnie z konwencją, w przypadku poprawnego zakończenia procesu zwracana jest wartość 0, a w przypadku błędu wartość niezerowa¹. Do oznaczania sukcesu czy porażki można użyć stałych: `EXIT_SUCCESS` i `EXIT_FAILURE`, zdefiniowanych w pliku `<stdlib.h>`. Wywołanie funkcji `exit` powoduje ponadto opróżnienie i zamknięcie wszystkich otwartych strumieni oraz usunięcie wszystkich tymczasowych plików utworzonych przy pomocy funkcji `tmpfile` (patrz `man tmpfile`). Można zdefiniować własne procedury zakończenia procesu i zarejestrować je przy pomocy funkcji bibliotecznych `atexit` i/lub `on_exit` (patrz podręcznik `man`). Takie procedury zostaną wywołane przez funkcję `exit` w kolejności odwrotnej do tej, w której zostały zarejestrowane. Pozwala to m.in. na opróżnienie wszystkich buforów standardowej biblioteki wejścia-wyjścia.

Pliki włączane	<stdlib.h>		
Prototyp	void atexit(void (*function)(void));		
Zwracana wartość	Sukces	Porażka	Czy zmienia errno
	0	≠ 0	Tak

Przedstawiona w powyższej tabeli funkcja `atexit` pozwala zarejestrować funkcję, która będzie wywołana przy normalnym zakończeniu procesu – albo poprzez wywołanie funkcji `exit`, albo przez `return` z funkcji `main`. Tak zarejestrowane funkcje są wywoływane w

¹Faktycznie zwracanych jest tylko osiem mniej znaczących bitów wartości `status`, zatem zwracane wartości należą do przedziału `[0,255]`.

kolejności odwrotnej do ich rejestracji; żadne argumenty nie są przekazywane. W funkcji zarejestrowanej przez `atexit` nie powinno się wywoływać funkcji `exit` – takie wywołanie jest niezdefiniowane w standardzie POSIX i może dawać różne efekty, zależne od implementacji. Typowo można zarejestrować do 32 funkcji, ale faktyczny limit zależy od implementacji (patrz `man atexit`). W ten sposób można automatycznie wywoływać funkcje, które robią określone porządki w momencie kończenia procesu. Oto prosty przykład użycia tej funkcji:

```
...
void koniec(void)
{
    printf("Koniec procesu - porzadki zrobione!\n");
}
...
int main(void)
{
    ...
    if (atexit(koniec) != 0) {
        perror("atexit error");
        exit(EXIT_FAILURE);
    }
    ...
}
```

Pliki włączane	<unistd.h>		
Prototyp	void _exit(int status);		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
			Nie

Funkcja `_exit` różni się od `exit` przede wszystkim tym, że jest *wywołaniem systemowym*, a nie funkcją biblioteki języka C. Powoduje ona natychmiastowe zakończenie procesu. Wszystkie otwarte deskryptory plików należące do procesu są zamykane, wszystkie jego procesy potomne są „adoptowane” przez proces `init` lub jego odpowiednik, a do procesu macierzystego wysyłany jest sygnał `SIGCHLD`. Znaczenie parametru `status` jest takie jak dla funkcji `exit`. W odróżnieniu do `exit` nie wywołuje ona żadnych procedur zarejestrowanych przez funkcje `atexit` lub `on_exit`. Natomiast to czy opróżnia standardowe bufory wejścia-wyjścia oraz czy usuwa pliki tymczasowe stworzone przy użyciu funkcji `tmpfile` jest zależne od implementacji. Generalnie zaleca się używanie funkcji `exit` w procesie macierzystym (za wyjątkiem przypadku tworzenia procesów *demonów*), natomiast funkcji `_exit` w procesach potomnych (by uniknąć efektów ubocznych).

2.4 Czekanie na procesy potomne – funkcja systemowa wait

W systemie UNIX na każdy proces, za wyjątkiem procesu `init` lub jego odpowiednika (o identyfikatorze `PID=1`), powinien czekać jakiś proces macierzysty. Proces, który się zakończył, ale na który nie czekał żaden inny proces nazywa się *zombie*. Proces *zombie* nic nie robi, ale zajmuje miejsce w systemowej tabeli procesów. Aby uniknąć powstawania procesów *zombie*, w Uniksie procesy „sieroty” są „adoptowane” przez proces `init`, który w odniesieniu do nich wykonuje operacje czekania (wówczas dla takiego procesu `PPID = 1`). W nowszych wersjach systemu Linux osierocone procesy potomne mogą być adoptowane przez specjalny proces *demon* o nazwie `systemd`, który może mieć `PID > 1` (sprawdzić poleceniem `ps tree -p`). Natomiast w systemie macOS proces ten nazywa się `launchd` i ma `PID = 1`.

Pliki włączane	<sys/types.h>, <sys/wait.h>		
Prototyp	pid_t wait(int *stat_loc);		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	PID procesu potomnego	−1	Tak

Do oczekiwania na proces potomny służy funkcja systemowa `wait`. Zawiesza ona działanie procesu macierzystego do momentu zakończenia się *pierwszego* procesu potomnego². Informacje o stanie potomka zwracane są przez parametr `stat_loc` (tylko dwa młodsze bajty są używane). Jeśli proces potomny zakończył się normalnie, to najmłodszy bajt będzie równy 0, a następny będzie zawierał kod powrotu. W przypadku zakończenia procesu potomnego na skutek sygnału, najmłodszy bajt będzie zawierał numer sygnału, a następny wartość 0 (w przypadku wygenerowania zrzutu pamięci *core*, najstarszy bit najmłodszego bajtu będzie ustawiony na 1). Istnieje kilka dedykowanych makr do wygodnego odczytu informacji o stanie zakończonego procesu zwracanej przez powyższy parametr. Na przykład makro `WIFSIGNALED(status)` zwraca wartość prawdy jeśli proces został zakończony przez sygnał, w takim przypadku makro `WTERMSIG(status)` zwraca numer odpowiedniego sygnału, gdzie `status` jest zmienną typu `int`, której należy przypisać wartość zwracaną poprzez parametr `stat_loc` funkcji `wait` (więcej informacji w `man 2 wait`). Gdy parametr funkcji `wait` będzie ustawiony na `NULL`, to stan procesu potomnego nie zostanie zwrócony. Jeżeli dany proces nie ma procesów potomnych, to funkcja kończy się błędem i ustawia zmienną `errno` na `ECHILD`.

Pliki włączane	<sys/types.h>, <sys/wait.h>		
Prototyp	pid_t waitpid(pid_t pid, int *stat_loc, int options);		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	PID procesu potomnego lub 0	−1	Tak

²UWAGA: Funkcja `wait` czeka na zakończenie tylko *jednego* procesu potomnego – tego, który zakończy się *najwcześniej*. W celu oczekiwania na zakończenie *kolejnego* procesu, trzeba ją wywołać *ponownie*.

Więcej możliwości niż funkcja `wait` dostarcza funkcja `waitpid`, np. można jej wskazać konkretny proces czy grupę procesów, na które ma czekać. Jeżeli argument `pid > 0` i argument `options = 0`, to funkcja zablokuje wywołujący ją proces do czasu zakończenia procesu potomnego o `PID = pid`. Znaczenie parametru `stat_loc` jest takie jak dla funkcji `wait`. Więcej szczegółów można znaleźć w podręczniku systemowym `man`.

2.5 Usypianie procesu – funkcja systemowa `sleep`

Użyteczną funkcją jest też funkcja `sleep`, która wstrzymuje („usypia”) wywołujący ją proces na wskazaną przez parametr `seconds` liczbę sekund. Kiedy upłynie podany czas uspiania procesu, to funkcja zwraca wartość 0, natomiast kiedy jej wykonanie zostanie przerwane (np. na skutek otrzymania sygnału), to zwraca liczbę nieprzespanych sekund.

Pliki włączane	<unistd.h>		
Prototyp	<code>unsigned sleep(unsigned seconds);</code>		
Zwracana wartość	Sukces	Porażka	Czy zmienia <code>errno</code>
	Liczba nieprzespanych sekund		Nie

Dostępne są też funkcje `usleep` i `nanosleep`, w których czas usypiania procesu/wątku można określać odpowiednio w mikrosekundach i nanosekundach (więcej informacji w podręczniku `man`).

ĆWICZENIE 1: PROCESY POTOMNE: `FORK`

- Napisać program wypisujący identyfikatory podane w tabeli 1 dla danego procesu.
- Wywołać funkcję `fork` trzy razy (najlepiej w pętli `for`) i wypisać powyższe identyfikatory dla procesu macierzystego oraz wszystkich procesów potomnych (najlepiej dla każdego procesu w pojedynczej linii). Przy pomocy funkcji `wait` sprawić, aby proces macierzysty zaczekał na zakończenie wszystkich procesów potomnych. → *Uwaga:* Nie używać funkcji `exit` ani instrukcji `return` w procesie potomnym.
- Jak w (b), tylko przy użyciu funkcji `sleep` (→ nie używać funkcji `wait`) sprawić by procesy potomne były adoptowane przez proces `init` lub jego odpowiednik. Następnie z poziomu powłoki wykonać polecenie `ps tree -p` i w wyświetlonym drzewie procesów zidentyfikować proces, który adoptuje osieroczone procesy w danym systemie.
- Jak w (b), tylko wstawić funkcję `sleep` w takich miejscach programu, aby procesy pojawiały się na ekranie grupowane pokoleniami od najstarszego do najmłodszego, a proces macierzysty kończył się dopiero po procesach potomnych (→ nie używać funkcji `wait`). Na podstawie wyników programu sporządzić „drzewo genealogiczne” tworzonych procesów, tzn. do pliku tekstowego „wkleić” wyniki programu, a następnie narysować drzewo genealogiczne, zaznaczając na nim identyfikatory `PID`, `PPID` i `PGID`.
- Jak w (b), tylko przy użyciu funkcji `setpgid` sprawić by każdy proces potomny stawał się liderem swojej własnej grupy procesów.
→ Ile procesów powstanie przy *n*-krotnym wywołaniu funkcji `fork` i dlaczego?

2.6 Uruchamianie programów – funkcja systemowa exec

Funkcja systemowa **exec** służy do ponownego **zainicjowania procesu** na podstawie **wskazanego programu**. Istnieje sześć odmian funkcji **exec** zgrupowanych w dwie rodziny (po trzy funkcje). Rodziny różnią się postacią argumentów: litera **l** w nazwie oznacza argumenty w postaci listy, a litera **v** – w postaci tablicy (ang. *vector*). Poniżej omawiamy po jednym przedstawicielu każdej z rodzin.

Pliki włączane	<unistd.h>		
Prototyp	<pre>int execl(const char *path, const char *arg0, ..., const char *argn, char *null); int execv(const char *path, char *const argv[]);</pre>		
Zwracana wartość	Sukces	Porażka	Czy zmienia errno
	Nic nie zwraca („popelnia samobójstwo”)	−1	Tak

Argumenty funkcji **exec**:

path ścieżkowa nazwa pliku wykonywalnego zawierającego program;
arg0 argument zerowy uruchamianego programu, na ogół jak wyżej;
arg1, ..., argn argumenty wywołania programu;
null wskaźnik zerowy dla typu **char**, tzn. (**char ***) **NULL**;
argv[] adres tablicy wskaźników na napisy (ang. *string*) będące argumentami przekazywanymi do wykonywanego programu (ostatnim elementem powinien być wskaźnik **null**).

Najczęściej funkcję **exec** wywołuje się w połączeniu z funkcją **fork**.

Typowe wywołanie **fork** i **exec**:

```
switch (fork())
{
    case -1:
        perror("fork error");
        exit(1);
    case 0: // proces potomny
        execlp("./nowy_program.x", "./nowy_program.x", (char *) NULL);
        perror("execlp error");
        _exit(2);
    default: // proces macierzysty (np. wywołanie funkcji wait)
};
```

ĆWICZENIE 2: URUCHAMIANIE PROGRAMÓW: EXEC

Zmodyfikować program z **ćwiczenia 1b** tak, aby komunikaty procesów potomnych były wypisywane przez odrębny program uruchamiany przez funkcję **execlp**. Nazwę programu do uruchomienia przekazywać przez argumenty programu procesu macierzystego.

→ Ile procesów powstanie przy *n*-krotnym wywołaniu funkcji **fork-exec** i dlaczego?